

Parameter-Efficient Fine-Tuning

Dhruv Kumar

BITS Pilani

Full fine-tuning: the baseline

Full fine-tuning updates **every** weight matrix in a pretrained model, using gradients computed from a smaller, labeled, task-specific dataset.

WHY THIS MOTIVATES PEFT

For a multi-billion-parameter model, updating and storing gradients for every weight puts fine-tuning out of reach on a single consumer GPU. PEFT methods avoid this by leaving almost all of the pretrained weights untouched.

What is PEFT?

Parameter-efficient fine-tuning (PEFT) adapts a pretrained model to a new task by training a **small fraction** of its parameters — often under 1% — while keeping the rest frozen.

PEFT methods fall into four families:

- **Additive** — insert new trainable modules into the network (e.g. *adapters*: small bottleneck layers between existing ones).
- **Selective** — train a subset of the existing parameters (e.g. *BitFit*: only the bias terms).
- **Reparameterization** — train a low-rank or otherwise compressed update to existing weights (e.g. **LoRA** — the focus of the rest of this lecture).
- **Soft-prompt** — prepend trainable vectors to the input or activations, freezing everything else (e.g. prefix-tuning, prompt tuning).

What is rank?

The **rank** of a matrix is the number of linearly independent rows it has (counting independent columns instead always gives the same number). It measures how much genuinely *new* information the matrix contains — a row that's just a scaled copy, or a sum, of other rows adds nothing new and doesn't raise the rank.

HOW TO COMPUTE IT: ROW REDUCTION

Use elementary row operations (subtract a multiple of one row from another) to reduce the matrix to echelon form. The rank is the number of **nonzero rows** left.

Example: $M = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$ — every row is a multiple of the first one:

$$\begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix} \xrightarrow[\substack{R_2 \rightarrow R_2 - 2R_1 \\ R_3 \rightarrow R_3 - 3R_1}]{} \begin{bmatrix} 1 & 2 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Only **1** nonzero row survives, so $\text{rank}(M) = 1$ — even though M is 3×2 with 6 entries. Compare the 3×3 identity I_3 : no row can be reduced away, so all 3 rows stay nonzero and $\text{rank}(I_3) = 3$, *full* rank for its size.

LoRA: low-rank adaptation

Instead of training the full update ΔW to a weight matrix, LoRA constrains it to be **low rank**: $\Delta W = BA$, a product of two thin matrices.

$$H = X \left(W_0 + \frac{\alpha}{r} BA \right)$$

$X \in \mathbb{R}^{n \times d}$ — input; $W_0 \in \mathbb{R}^{d \times d_{out}}$ — frozen pretrained weight; $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times d_{out}}$ — the only trainable matrices, rank $r \ll \min(d, d_{out})$; α — a fixed scaling constant (tuning α behaves like tuning a learning rate).

TWO DETAILS THAT MAKE IT WORK

B is initialized to **all zeros** (and A to small random values), so $\Delta W = 0$ and training starts exactly at the pretrained model. After training, BA can be **merged** directly into W_0 — inference has zero added latency.

LoRA: a numeric example

Take $d = 3$, $d_{out} = 2$, rank $r = 1$, $\alpha = 2$ (scale $\alpha/r = 2$):

$$W_0 = \begin{bmatrix} 2 & 1 \\ 0 & 1 \\ 1 & 2 \end{bmatrix} \quad B = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \quad A = \begin{bmatrix} 1 & 2 \end{bmatrix}$$

$$\Delta W = BA = \begin{bmatrix} 1 & 2 \\ 1 & 2 \\ 0 & 0 \end{bmatrix} \quad W' = W_0 + \frac{\alpha}{r} \Delta W = \begin{bmatrix} 4 & 5 \\ 2 & 5 \\ 1 & 2 \end{bmatrix}$$

SEPARATE B,A VS. MERGED W' AGREE EXACTLY

For input $x = [1, 1, 1]$: through B, A separately, $h = xW_0 + \frac{\alpha}{r}(xB)A = [3, 4] + 2[2, 4] = [7, 12]$. Through the merged W' directly, $h = xW' = [7, 12]$ – identical. This is why merging costs nothing at inference: the two are algebraically the same computation.

LoRA: verified parameter savings

Take one 4096×4096 projection matrix (a typical hidden size) with LoRA rank $r = 8$:

$$d \times d = 4096 \times 4096 = 16,777,216 \quad \text{vs.} \quad r(d + d) = 8 \times 8192 = 65,536$$

That's a **256×** reduction in trainable parameters for one matrix — and the same ratio holds at full-model scale, since every matrix shrinks by the same factor.

Apply LoRA to all four attention projections (W_Q, W_K, W_V, W_O) in every layer of a 32-layer, $d = 4096$ model (LLaMA-7B-shaped, 6.74B total parameters):

$$32 \times 4 \times 65,536 = 8,388,608 \text{ trainable params}$$

That's just **0.124%** of the full 6.74B-parameter model — trained, instead of frozen.

Reading list

START HERE

[Lialin et al., "Scaling Down to Scale Up: A Guide to PEFT" \(2023\)](#)

[Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models" \(2021\)](#)

OTHER PEFT FAMILIES

[Houlsby et al., "Parameter-Efficient Transfer Learning for NLP" \(2019\)](#) — adapters

[Zaken et al., "BitFit" \(2021\)](#)

[Li & Liang, "Prefix-Tuning" \(2021\)](#)

[Lester et al., "The Power of Scale for Prompt Tuning" \(2021\)](#)

COMING UP NEXT

QLoRA, DoRA, and AdaLoRA — three ways to push LoRA further, covered in the next lecture.